Testing Guide 03: every test in Waypoint, and why it exists

There are about 3,000 automated checks in this repository. This guide explains:

- what kinds there are, and why each kind is needed;
- which ones matter most, and which may be redundant;
- how the main flow, the edge cases, security, authorization, the AI paths and the frontend-to-backend connection are tested;
- which engineering principles the tests rest on (SOLID, CAP, ACID, the test pyramid and others);
- where the gaps are.

Written against `main` at `d086270` on 2026-10-16. Counts come from the code on that date.

Companions: AI and System Guide 01 and Spring Boot Guide 02.

Contents

1. Why test at all
2. The whole inventory
3. The kinds of test, and why each is needed
4. The principles behind the tests
5. The most critical tests
6. The main flow, end to end
7. Edge cases: the register and how each is handled
8. Security, authentication and authorization tests
9. Concurrency, idempotency and offline tests
10. AI, ML and tool-call (MCP) tests
11. Frontend tests
12. How frontend and backend are tested together
13. Test doubles: fakes, stubs and recorders
14. Redundant, slow or misplaced tests
15. Gaps
16. How to run them
17. Writing a good test here

---

1. Why test at all

Waypoint decides real things: which store gets its fresh goods before 08:00, who may see whose orders, and what a driver recorded offline in the hills. A bug can cost:

- a store its morning;
- a driver a lost delivery record;
- the business a data leak.

The tests exist to make five things true **before** a change reaches anyone:

1. **The rules hold.** A planning constraint, a state transition or a permission behaves as written in `RULES-AND-POLICIES.md` (292 rules, each with an id the tests cite).
2. **The edge cases are handled.** Every one of the 388 cases in `EDGE-CASES.md` names its test: "a case without a test is a case that is not handled".
3. **The architecture stays intact.** Module boundaries, layer purity and naming are checked by tests, not by memory.
4. **Concurrency and replays are safe.** Two people, two replicas or one offline phone sending twice never corrupt data.
5. **A change can be made without fear.** With the suite green, someone new can refactor a 1,000-line planning class and know what they broke.

Without tests, every one of these depends on a reviewer noticing. With 1,278 backend test methods, that is not possible by eye.

---

2. The whole inventory

Where	Files	Tests	Runner	Speed
Backend unit (domain, policies, parsers)	121 classes	733 methods	JUnit 5 under Maven surefire	milliseconds each
Backend integration (real PostgreSQL, full Spring context)	57 classes	544 methods	JUnit 5 + `@SpringBootTest` or a module support base	seconds each
Backend architecture (part of the unit count)	`ModuleBoundaryTest`, `EventCatalogueTest`, `SeededPolicyTest`	20	ArchUnit, JUnit	seconds
Frontend Node (logic, wording, boundaries, service worker)	83 files	407	`node --test`	milliseconds
Browser, shell (`tests/e2e`)	9 specs	22	Playwright, Chromium	minutes
Browser, dispatcher	23 specs	104	Playwright	minutes
Browser, driver	14 specs	30	Playwright	minutes
Browser, loader	12 specs	31	Playwright	minutes
Browser, store	22 specs	62	Playwright	minutes
Model service (`ml-server/tests`)	3 files	24	pytest	seconds
MCP server (`mcp/tests`)	5 files	29	node test	seconds
Alert rules (`observability/prometheus/tests`)	1 file	all 18 alerts	`promtool test rules`	seconds
Official Task 2B validator	`tools/check_allocation`	1 run	Python, in CI	seconds

In total: about 1,278 backend, 407 Node, 249 browser, 24 Python and 29 MCP tests, plus the alert rules and the validator. Many browser tests run on several screen sizes (`devices.spec.ts`), so the number of executions is higher.

Backend tests by module, as class counts:

Module	Classes
planning	31
platform	24
intelligence	21
identity	21
execution	14
issues	9
top-level cross-module tests	9
ordering	8
receipt	7
referencedata	6
notification	6
messaging	5
warehouse, loading, demo	4 each
architecture, shared	2 each
sync	1

3. The kinds of test, and why each is needed

```
                    /\        browser (Playwright): 249     slow, few, whole screens
                   /  \
                  /----\      integration (real DB): 544    seconds, every command
                 /      \
                /--------\    architecture + contract       rules about the code itself
               /          \
              /------------\  unit (pure domain): 733 + 407 Node   fast, many, every rule
             /______________\
```

This is the **test pyramid**. Many fast tests check the rules; fewer slow ones check that the pieces are wired. Each level catches what the one below cannot.

3.1 Unit tests (domain)

- **What:** one class, no database, no Spring, the clock passed in. For example `PolicyEvaluatorTest`, `PinPolicyTest`, `PublicationGateTest`, `PromptFactsTest` and `CircuitBreaker` tests.
- **Why:**
  - They are the cheapest way to check every branch of a rule, so they can be exhaustive.
  - They run in milliseconds, so they run on every save.
- **Possible only because** of architecture rule 2: "`domain/` is pure. Time and randomness are parameters." A rule that cannot be tested without a database is in the wrong layer.
- **Required for:** every rule change (AGENTS.md).

3.2 Integration tests

- **What:** the whole Spring application against a **real PostgreSQL** (a dedicated `TEST_DATABASE_URL`, or a throwaway container). They go through the command bus, transactions, row-level security, the outbox and the version guards. For example `OrderingSchemaIntegrationTest`, `CommandPathIntegrationTest` and `IdentityHardeningIntegrationTest`.
- **Why:**
  - Most serious bugs here live **between** the pieces: a migration that forgot a grant, an RLS policy that hides too much, a `SET LOCAL` that was not set, two transactions that conflict.
  - A mocked database cannot catch any of those.
- **Required for:** every command, with **one denied-scope case** each.

3.3 Authorization tests

- **What:** for every command and read, at least one case where someone outside scope tries it and gets `403` plus an audit row.
- **Why:** "Deny by default" is a security property. Its failure mode is silent: an empty list instead of a refusal, or a leak instead of a `403`. Only a test that tries it proves it.

3.4 Concurrency tests

- **What:** real parallel threads against the real database, for example "two parallel amends of one version let exactly one win" or "two relays never claim the same event".
- **Why:** race conditions do not show up in sequential tests. The version guard, `FOR UPDATE SKIP LOCKED`, advisory locks and PIN-attempt counting are all claims about parallel behaviour.

3.5 Architecture tests

- **What:** ArchUnit reads the compiled classes. `ModuleBoundaryTest` has 9 rules: modules reach each other only through `contract`, `domain` is framework-free, only the platform seam touches JDBC, no class is named `*Service`, and others. `EventCatalogueTest` keeps the event records equal to MODULES.md. `SeededPolicyTest` keeps the seeded role policies as designed.
- **Why:** structure decays one "just this once" import at a time. A test makes the rule a build failure instead of a review comment.

3.6 Contract tests

- **What:**
  - `ProblemContractTest` checks that every error is RFC 9457 `problem+json` with `violations`;
  - `CommandPathIntegrationTest.everyHandlerHasACatalogueRowThatSaysItIsImplemented` checks that every command kind is in the IAM catalogue;
  - `EventCatalogueTest`.

- **Why:** clients branch on the error body and the command kinds. Changing them silently breaks phones that are offline and replay later.

3.7 Browser tests (Playwright)

- **What:** a production build of the frontend in Chromium, against a **mocked API** (`mocks.ts` beside each role's specs). One suite per role plus the shell.
- **Why:**
  - Offline behaviour lives in the browser (IndexedDB queue, service worker, reload, sync), and so do accessibility, layout on each device and the user's flow through screens.
  - A mocked API makes them fast and deterministic; the backend's behaviour is proven separately by the integration tests.
- **Required for:** offline changes (offline save, reload, sync, conflict).

3.8 Node tests (frontend logic)

- **What:** the role logic that is not a screen: the offline queue's retry and lanes, wording formats, the boundaries between frontend folders, the service-worker cache rules, reply ranking, photo encoding.
- **Why:** this logic is too detailed to check through a browser, and too important to leave unchecked.

3.9 Model, alert and validator tests

- **Model service (pytest):**
  - the service must give exactly the same answers as the original Datathon inference code;
  - the leakage and retraining tools are tested;
  - the service refuses to start on a model file whose SHA-256 differs.
- **Alert rules (`promtool test rules`):** each of the 18 alerts fires on the series it should, and stays quiet on the rest.
- **Official Task 2B validator:** CI runs the competition's own checker on the engine's peak-day output (`PeakDayAllocationTest` writes it).

3.10 Model evaluation (not tests)

Measuring a model's quality is not a pass/fail test. It runs on live data, every day, in the product:

- labels;
- Brier score, log loss, calibration;
- backtest, drift (PSI), the leakage check.

Guide 01 section 9.4 covers it. The tests check that the evaluation **code** is right (`BacktestTest`, `ModelMetricsTest`, `LateThresholdTest`); the evaluation itself says whether a model is good.

---

4. The principles behind the tests

4.1 Testing principles

Principle	Meaning	How Waypoint applies it
Test pyramid	many fast low-level tests, few slow high-level ones	1,140 unit and Node, 544 integration, 249 browser
F.I.R.S.T.	Fast, Independent, Repeatable, Self-validating, Timely	the clock is injected, so tests are repeatable; each integration test makes its own accounts with a random suffix, so tests are independent; assertions are `assertEquals`, not printouts
Arrange, act, assert (given, when, then)	set up, do one thing, check	every test method; names read as the sentence of what must be true
Test behaviour, not implementation	assert what a user or caller sees	`aStaleVersionIsAConflictAndARetriedCommandIsAnsweredOnce` checks the answer and the database, not which method was called
Names as specifications	the name is the requirement	`theLastActiveDispatcherOfADepotCannotBeDisabledDemotedOrTakenOffIt`
Real dependencies where they matter	do not mock what you are trying to prove	PostgreSQL is real in integration tests, because RLS, grants and isolation are the thing under test
Doubles at the edges only	fake what is outside the process	`FakeS3`, `StubWarehouse` and the model service stub (section 13)
One rule, one place, one test	the constraint registry is read by the engine, the gate and the UI	tested once in `ConstraintsTest`, then trusted by its callers
A case without a test is not handled	the edge-case register lists a test per case	section 7
Determinism	no real time, no unseeded randomness	`Clock` parameters; `IssueTriage.shuffled(seed)`; seeded synthetic days in planning tests

4.2 SOLID makes the code testable

Principle	How it helps the tests
Single responsibility	one handler per command, so one test class per behaviour
Open/closed	a new command adds a handler and its tests; old tests do not change
Liskov substitution	a `FakeS3` or `StubWarehouse` can stand in for the real adapter, because each port's contract is honoured
Interface segregation	small ports (`LanguageModel` has 3 methods), so a fake is a few lines
Dependency inversion	the domain depends on interfaces, so unit tests need no Spring and no database

4.3 ACID, CAP and consistency, as the tests see them

ACID (one PostgreSQL transaction):

- **Atomic.** `OutboxRelayIntegrationTest.anEventFromARolledBackTransactionIsNeverDelivered`: state, receipt, audit and event commit together or not at all.
- **Consistent.** The version guard and the domain's state machine. `OrderStateMachineTest` reject illegal moves.
- **Isolated.** Serializable isolation. `twoParallelAmendsOfOneVersionLetExactlyOneWin`.
- **Durable.** PostgreSQL itself. Backups are a gap (section 15).

CAP theorem. A distributed system that is split by a network fault must choose between **consistency** (refuse to answer rather than give a wrong answer) and **availability** (answer, possibly with stale data). Waypoint makes a different choice per role, and the tests check each choice:

Part	Choice	Why	Tested by
The server: one PostgreSQL	**C** (consistent)	one database, serializable; a write is refused rather than merged	version-guard and concurrency integration tests
Driver offline	**A** (available)	the driver keeps recording with no signal; conflicts are held for a person to decide, never merged automatically	`SyncIntegrationTest`, `aStopRecordedOfflineAndReplayedThroughSyncAppliesOnce`, driver offline browser specs

Part	Choice	Why	Tested by
Loader and store	mostly C, with kept reads	they read offline but most writes wait for a connection	`kept-reads.test.ts`, the loader and store offline specs
Dispatcher	**C** only	two dispatchers planning offline would produce plans that cannot be reconciled, so offline planning is refused	`tiers.ts`, dispatcher specs
Between modules	**eventual consistency**	events through the outbox arrive after the commit, at least once, in order per aggregate	`OutboxRelayIntegrationTest`, the consumer integration tests

Idempotency is what makes "at least once" safe: the same command or event applied twice equals once. Section 9 lists the tests.

---

5. The most critical tests

If you could keep only these, keep these. Each guards something whose failure is a security breach, data loss or a wrong delivery.

#	Test	Guards	Why it is critical
1	`CommandPathIntegrationTest`	the single write path: applies once however often retried; same id with a different payload rejected; unknown kind forbidden; driver denied and recorded; every handler has a catalogue row	every write in the system goes through this path
2	`CommandBusTest`, `IdempotencyGuardTest`	fail closed with no authorizer; replay; fingerprint	a bug here makes the whole system either open or unusable
3	`PolicyEvaluatorTest`, `PolicyCacheTest`, `SeededPolicyTest`	default deny, explicit deny wins, the role policies, cache cleared on change	permissions for everyone
4	`IdentityHardeningIntegrationTest` (25 tests)	lockout, cookie, session expiry, re-check inside the transaction, role and scope changes end sessions, pool runs as `waypoint_app`, schema owner is not a superuser, RLS blocks a hidden backfill, driver scope by date	the security foundation
5	`FoundationIntegrationTest`	migrations apply and are idempotent; Argon2 not plaintext; RLS returns nothing with no actor; disabling an account revokes sessions at once; a policy with an unknown action is rejected	the platform promises
6	`ModuleBoundaryTest`	the architecture	without it the modular monolith decays into a big ball of mud
7	`OutboxRelayIntegrationTest` (13 tests)	no event from a rolled-back transaction; one delivery per consumer; two relays never claim the same event; order per aggregate; dead-lettering; lease recovery	every cross-module effect depends on it
8	`ConstraintsTest`, `PublicationGateTest`, `PlanningRunTest`, `ScarceFleetReplanTest`, `PeakDayAllocationTest` + the official validator	no plan breaks a constraint; a draft on old demand or rules cannot publish; the Task 2B peak day	the core business decision
9	`OrderingSchemaIntegrationTest`, every `*SchemaIntegrationTest`	who sees which row, through RLS	data leaks between stores and depots
10	`SyncIntegrationTest`, the driver offline specs	offline records replay once, conflicts held	lost or doubled deliveries
11	`ReceiptHandoverIntegrationTest`	the handover PIN: returned once, only a hash kept, parallel wrong entries never exceed five guesses	proof that goods were handed over
12	`PromptFactsTest`, `IntelligenceIntegrationTest` (AI cases)	only allowed, masked facts leave; an unfaithful answer is thrown away; competition models gated	personal data and AI safety
13	`McpEnterpriseIntegrationTest`, `McpOAuthIntegrationTest`, `McpRateLimitIntegrationTest`	assistant scopes, confirmed writes, PKCE, rate limits	an AI assistant acting as a person
14	`wording.test.ts`, `boundaries.test.ts`	the words people read; frontend structure	consistency across roles

6. The main flow, end to end

The day is: **order, then plan, load, deliver, receive**. Each step is tested at three levels:

- the **domain**: is the rule right?
- **integration**: is it wired, scoped and versioned?
- the **browser**: can the person do it on screen?

Step	Domain (unit)	Integration (real DB)	Browser
1. Store places an order before the 16:00 cutoff	`CutoffAndDeliveryDateTest`, `OrderStateMachineTest`	`OrderingCommandIntegrationTest` (placement after the cutoff rolls to the next run), `OrderingSchemaIntegrationTest` (only that outlet's manager and the depot see it)	`e2e-store/order.spec.ts`, `orders.spec.ts`
2. Warehouse reserves stock	`WarehouseDomainTest`	`WarehouseIntegrationTest` (a placement is confirmed once however often it is sent; signature checks)	(no browser: an external system)
3. Dispatcher generates and decides a plan	`ConstraintsTest`, `PlanningRunTest`, `ScarceFleetReplanTest`, `CostReplanTest`, `PriorityPolicyTest`	`PlanGenerationQueueIntegrationTest` (two dispatchers pressing at once get the same job), `PlanningDecisionsIntegrationTest`, `PlanningRevisionIntegrationTest`	`e2e-dispatcher/plan.spec.ts`, `plan-decisions.spec.ts`, `decisions.spec.ts`
4. Publish	`PublicationGateTest`, `PublishedEditWindowTest`	`PlanningCommandIntegrationTest`, `PlanningSchemaIntegrationTest`	`plan-published.spec.ts`, `plan-risk.spec.ts`, `plan-risk-review.spec.ts`
5. Loader loads the vehicle	`LoadingSession` tests	`LoadingIntegrationTest` (simultaneous checks cannot both consume one version; a queued check replayed applies once)	`e2e-loader/release.spec.ts`, `held.spec.ts`, `offline.spec.ts`
6. Driver runs the trip, offline	`EtaPolicy`, `PositionPolicyTest`, delivery record tests	`ExecutionIntegrationTest`, `SyncIntegrationTest` (a stop recorded offline and replayed through sync applies once)	`e2e-driver/day.spec.ts`, `redo.spec.ts`, `location.spec.ts`, `position-lane.spec.ts`
7. Store receives with the handover PIN	`Handover`, `Receipt` tests	`ReceiptHandoverIntegrationTest`	`e2e-store/receive.spec.ts`, `handover.spec.ts`; `e2e-driver/handover.spec.ts`
8. Issues and messages throughout	`Issue`, `DecisionRules`, `MessagePolicy` tests	`IssuesCommandIntegrationTest`, `IssuesSchemaIntegrationTest`, messaging integration tests	`e2e-*/issues.spec.ts`, `messages.spec.ts`, `trip-messages.spec.ts`
9. Notifications	`NotificationPolicy` tests	notification integration tests	`notifications.spec.ts` in each role

One full day in one place.

- `PeakDayAllocationTest` plans the Task 2B peak day through the real engine, and the official validator checks its CSV.
- The demo control room's `DemoIntegrationTest.resetPreparesOnlyAnEmptyOperatingDayThroughOwnerCommands` prepares a day through the same commands people use.
- **What is not automated:** a single test that walks all nine steps against a running backend. The README walk is done by hand (section 15).

---

7. Edge cases: the register and how each is handled

7.1 The register

`docs/architecture/EDGE-CASES.md` has **388 cases**. Each row has six columns:

- **ID;**
- **trigger:** what happens;
- **required behaviour:** what the system must do;
- **enforced in:** which layer;
- **detection:** the metric or alert that shows it in production;
- **test:** which test proves it.

Prefix	Area	Cases
SEC	identity, access, security	49
PLN	planning and capacity	47
EXE	execution, the road, offline	46
ML	intelligence	37
MSG	messaging	27
ORD	ordering and demand	24
RCP	receipt and dispute	23
ISS	operational issues	21
PLT	platform, data, time	20
NOT	notification	15
STK	stock and the warehouse	14
POL	policy and rule change	10
LOD	loading and the dock	9
REF, DEMO	reference data, demo mode	8 each
CAT	catalogue	6
FLT	fleet	5
FLOW	cross-module flows	3
EXE-LOC, IAM-OFF and others	live location, offline PIN	the rest

Coverage of the register.

Kind of entry	Cases
Name one or more specific tests	**342**
Name only a kind ("Unit", "Integration", "Migration test") without the test	23
Have no automated test: a gap, a follow-up, a chaos drill, a load test or manual	23

Section 15 lists the second and third groups.

7.2 The nine handling patterns

Most cases are one of nine patterns (EDGE-CASES §1), and each pattern has its kind of test:

Pattern	Example case	How it is tested
Idempotent replay	a retried command, a redelivered event	send it twice; assert one effect and the same answer
Version guard	two edits of one record	send a stale `expectedVersion`; assert `409` and no change
Human-in-the-loop conflict	an offline write that collided	replay through sync; assert it is held, not merged (`SyncIntegrationTest`, `sync-review.test.ts`)

Pattern	Example case	How it is tested
Degrade visibly	the model service is down	stop the dependency; assert the fallback **and** the message on screen (`withTheModelServiceDownThePlanIsScoredDeterministicallyAndSaysSo`)
Circuit breaker	a provider keeps failing	inject failures with a test clock; assert open, half-open, closed transitions
Retry with backoff and jitter	a transient failure	`RelayBackoffTest`, `offline-retry.test.ts`
Compensating action	something committed must be undone	assert a new attributed record, never an `UPDATE` that hides the original
Escalation timer	waiting on a human	advance the clock; assert the reminder fires (the reminder cases in `AttentionPolicyTest`)
Deny by default	any authorization question	try out of scope; assert `403` and the audit row

7.3 Examples, from trigger to test

Case	Trigger	Behaviour	Test
ORD-01	an order after 16:00	accepted for the next operating run, by the server clock in Colombo	`CutoffAndDeliveryDateTest`, `OrderingCommandIntegrationTest.aPlacementAfterTheCutoffRollsToTheNextRun`
ORD-10	a store cancels after loading started	refused	`OrderStateMachineTest` (`LOADING` has no `CANCELLED` edge)
PLN-06	two dispatchers edit one draft	one wins on the row version	`DraftEditHandler` path, planning revision tests
SEC-03	a permission revoked while a command waits	the re-check inside the transaction refuses	`IdentityHardeningIntegrationTest.aPermissionRevokedMidFlightIsCaughtByTheReCheckInsideTheTransaction`
EXE-LOC-06	two position batches overlap	both land, no duplicates	`ExecutionIntegrationTest.twoBatchesForOneVehicleAtOnceBothLandWithoutDuplicates`
PLT-13	events of one order across a failure	arrive in write order	`OutboxRelayIntegrationTest.oneAggregatesEventsArriveInWriteOrderEvenAcrossAFailure`

8. Security, authentication and authorization tests

Authentication.

What	Tests
Password stored as Argon2, never plaintext	`FoundationIntegrationTest.passwordsAreStoredAsArgon2NotPlaintext`
Wrong password reveals nothing	`signingInRequiresTheRightPasswordAndRevealsNothingOtherwise`
8 failures lock the identity; the owner can still sign in from elsewhere; the lockout lifts	`IdentityHardeningIntegrationTest` (four tests), `LockoutPolicyTest`, `LoginAuditTest`
Cookie strict, lasting as long as the session, stored only as a hash	`theSessionCookieIsStrictLastsAsLongAsTheSessionAndIsStoredOnlyAsAHash`, `SessionCookieTest`
Session expires by the clock, touched at most once a minute	`aSessionExpiresByTheClockAndIsTouchedAtMostOnceAMinute`
Signing in again ends the previous session	`signingInAgainEndsTheSessionTheBrowserAlreadyHeld`
Disabling an account revokes sessions at once	`FoundationIntegrationTest.disablingAnAccountRevokesItsSessionsAtOnce`
Loader PIN: 5 tries, pause; parallel wrong entries never exceed five	`PinPolicyTest`, `parallelWrongEntriesNeverGiveMoreThanFiveGuesses`, `OfflineOperatorTest`, `loader-offline-pin.test.ts`
OAuth for assistants: PKCE, code reuse refused, concurrent exchange cannot issue two tokens	`McpOAuthIntegrationTest`, `OAuthRulesTest`
Demo sign-in offers only working, non-privileged accounts, only in demo mode, until its date	`DemoSignInIntegrationTest`, `e2e/demo-sign-in.spec.ts`

Authorization. 212 backend test methods are about scope, denial or refusal.

What	Tests
Default deny, explicit deny wins, wildcards	`PolicyEvaluatorTest`
Seeded role policies decide differently per role	`SeededPolicyTest`, `seededRolePoliciesDecideDifferently`
A policy change takes effect at once; the cache clears	`anAdminChangesAPolicyAndTheDecisionChangesImmediately`, `PolicyCacheTest`
An unknown action is rejected when written	`aPolicyNamingAnUnknownActionIsRejectedWhenWritten`
Every module: out of scope is `403` and audited, never empty	one denied case per command, for example `aListOutsideScopeIsForbiddenAndAuditedNeverEmpty`, `aDriverIsDeniedAndTheDenialIsRecorded`, `aManagerCannotWriteAnOrderForAnotherOutlet`
Row scope: who sees which row	every `*SchemaIntegrationTest`, `theOutletsManagerSeesTheOrderAndNoOneElseDoes`
Driver scope holds on assigned dates only	`driverScopeHoldsOnTheAssignedDatesAndFailsAcrossTheBoundary`
Live streams refuse another depot	`PlanGenerationQueueIntegrationTest.theLiveBoardStreamOpensForTheDispatchersDepotAndRefusesAnother`, `ChangeSignalsTest`

Database protection.

What	Tests
The pool runs as `waypoint_app` and cannot touch a table without a module role	`thePoolRunsAsWaypointAppAndCannotTouchATableWithoutAdoptingARole`
The schema owner is not a superuser; a migration cannot do what only a superuser can	`theSchemaBelongsToAnOwnerThatIsNotASuperuser`, `migrateCanLogInAsTheOwnerItselfWhichCannotDoWhatASuperuserCan`
A new table gets its module's grants	`aLaterMigrationRunsAsTheOwnerAndItsTableGetsTheModulesGrants`
A backfill RLS would hide fails instead of silently doing nothing	`aBackfillThatRowLevelSecurityWouldHideFailsAndChangesNothing`
RLS returns nothing with no actor	`rowLevelSecurityReturnsNothingWhenNoActorIsSet`

The web edge.

What	Tests
A state change from another origin is refused	`OriginGuardFilterTest`, `aStateChangingRequestFromAnotherOriginIsRefused`
Errors are problem+json, never driver text	`ProblemContractTest`
Correlation ids	`CorrelationIdFilterTest`
What may be cached	`CacheablePathsTest`, `ConditionalReadsIntegrationTest`, `sw-cache.test.ts`
Audit snapshots carry no personal data	`AuditRedactorTest`, `AuditApiIntegrationTest`
Object encryption	`ObjectCipherTest`, `ObjectStoreIntegrationTest`, `SigV4Test`
Content-Security-Policy	`e2e/shell.spec.ts` (`the app shell loads under its Content-Security-Policy`)

9. Concurrency, idempotency and offline tests

Concurrency (real parallel threads, `CountDownLatch` or thread pools, against the real database):

Test	Proves
`OrderingCommandIntegrationTest.twoParallelAmendsOfOneVersionLetExactlyOneWin`	the version guard under a race
`PlanGenerationQueueIntegrationTest.twoDispatchersPressingAtOnceGetTheSameJob`	one plan generation per press, not two
`LoadingIntegrationTest.simultaneousChecksCannotBothConsumeTheSameVersion`	dock checks
`AdministrationIntegrationTest.twoDriversCannotHoldOneVehicleOverTheSameDays`	assignment overlaps
`SchedulerIntegrationTest.twoRunnersOnOneJobRunItOnceAndCountTheDuplicate`	one cutoff, even with two replicas
`OutboxRelayIntegrationTest.twoRelaysNeverClaimTheSameEvent`	`SKIP LOCKED`
`McpOAuthIntegrationTest.replayRevokesTheFirstTokenAndConcurrentExchangeCannotIssueTwoTokens`	OAuth code exchange
`ReceiptHandoverIntegrationTest.parallelWrongEntriesNeverGiveMoreThanFiveGuesses`	the PIN limit under a race
`ExecutionIntegrationTest.twoBatchesForOneVehicleAtOnceBothLandWithoutDuplicates`	position batches
`execution/LiveReadLoadTest`	the Live read at 1,280 trips

Idempotency and replay:

- 65 test methods are about replays.
- 55 are about stale versions and conflicts.
- For example:
  - `aCommandAppliesOnceHoweverOftenItIsRetried`;
  - `aRedeliveryAppliesOncePerConsumer`;
  - `aReservedPlacementIsConfirmedOnceHoweverOftenItIsSent`;
  - `replayingAPublishedVersionDoesNotResetWorkOrAdvanceTheVersion`.

Offline (the driver's full tier, the loader and store resilient tiers):

Layer	Tests
Queue logic	`offline-lanes.test.ts`, `offline-retry.test.ts`, `offline-outcome.test.ts`, `offline-retention.test.ts`, `offline-position-backlog.test.ts`, `offline-waits-for.test.ts`, `kept-reads.test.ts`, `sync-review.test.ts`
Service worker	`sw-cache.test.ts`, `sw-drain.test.ts`, `sw-f05.test.ts`, `e2e/background-sync.spec.ts`
Browser	`e2e-driver/redo.spec.ts`, `position-lane.spec.ts`, `request-rate.spec.ts`; `e2e-loader/offline.spec.ts`, `offline-pin.spec.ts`; `e2e-store/offline.spec.ts`, `kept-state.spec.ts`
Server side	`SyncIntegrationTest`, `aStopRecordedOfflineAndReplayedThroughSyncAppliesOnce`, `aQueuedCheckReplayedThroughSyncAppliesOnce`

10. AI, ML and tool-call (MCP) tests

The AI rule is that a model may advise but never decide, and must fail safe. The tests check the **guardrails**, not whether a model is clever. A model's quality is measured by evaluation (section 3.10).

The language model gateway (explanations):

Guardrail	Test
Only fields on the allow-list leave; unknown shapes are dropped	`PromptFactsTest.onlyFieldsOnTheListGoAndUnknownShapesAreDropped`
Phones, identity numbers, e-mails and links are masked	`PromptFactsTest.phonesIdentityNumbersEmailsAndLinksAreMaskedBeforeAnythingLeaves`
An answer with a figure the facts do not hold is thrown away	`PromptFactsTest.anAnswerWithAFigureTheFactsDoNotHoldIsThrownAway`
Through the gateway, allowed and masked facts only, asked once per plan, recorded	`IntelligenceIntegrationTest.aPlanIsWordedThroughTheGatewayWithOnlyAllowedMaskedFactsAndOnce`
Config refuses AI on with no key, or Jev on `latest`	`AiProperties` validation (startup)
The screen shows "rules only" and the AI-worded text	`dispatcher-explain.test.ts`, `e2e-dispatcher/explain.spec.ts`

Jev (shadow):

Guardrail	Test
Every issue type is an option, in an order fixed per issue but not the same for all (against first-option bias)	`IssueTriageTest.everyIssueTypeIsAnOptionInAnOrderFixedPerIssueButNotTheSameForAll`
Only real words are asked about, and they are masked	`IssueTriageTest.onlyRealWordsAreAskedAboutAndTheyAreMasked`
The guess is kept beside the real type, from masked words	`IntelligenceIntegrationTest.jevGuessesAPersonsIssueFromItsMaskedWordsAndTheGuessIsKeptBesideTheRealType`

The Datathon models and the gate:

Guardrail	Test
A published plan is scored by the active model and stamped with it	`aPublishedPlanIsScoredByTheActiveModelAndStampedWithIt`
Model service down: deterministic answer, and says so	`withTheModelServiceDownThePlanIsScoredDeterministicallyAndSaysSo`, `withTheModelServiceDownTheForecastIsTheWeekdayMeanAndSaysSo`
A server with another version is refused, with the reason	`aServerWithAnotherVersionIsDegradedWithTheReason`
A degraded plan is re-scored once the model is back	`aDegradedPlanIsRescoredOnceTheModelIsBack`
Only an administrator activates a model; a retired model cannot come back	`aDispatcherCannotActivateAModel`, `aRetiredModelKeepsItsReasonAndCannotBeActivatedAgain`
Each stop's prediction says why, in plain reasons	`eachStopsPredictionSaysWhyItIsAtRiskInPlainReasons`, `LateReasonsTest`
The training export is the administrator's alone	`theTrainingExportIsTheAdministratorsAlone`
The service equals the vendored Datathon inference; refuses a changed model file	`ml-server/tests/test_service.py`
Leakage check and retrain gate	`test_leakage.py`, `test_retrain.py`

The evaluation code:

Check	Test
Backtest judges against the estimate, on enough labels	`BacktestTest` (three cases)
Late threshold frozen by the Wilson lower bound	`LateThresholdTest`
A missing arrival is no label, never "not late"; one outcome per stop	`OutcomeLabelTest`, `anOutcomeIsRecordedOnceAndADeliveryNeedsAnArrival`
Daily metrics from labelled outcomes only	`ModelMetricsTest`, `eachModelsDayIsMeasuredFromItsLabelledOutcomesOnly`

Check	Test
Reply ranker: rule order below the minimum; position pull learned apart	`ReplyRankerTest`
Model alerts fire correctly	`observability/prometheus/tests/waypoint.test.yml`

The watcher (attention watch):

Check	Test
An item raised once, refreshed, cleared	`anAttentionItemIsRaisedOnceRefreshedAndThenClearedWhenItNoLongerHolds`
Rules per stop	`AttentionPolicyTest`, `AttentionWatchMappingTest`
Reminders	the reminder cases in `AttentionPolicyTest`

Tool calls (MCP), where an assistant acts as a person:

Guardrail	Test
A narrow scope allows only its area and names the rule when it refuses	`McpEnterpriseIntegrationTest.aNarrowScopeAllowsOnlyItsAreaAndNamesTheRuleWhenItRefuses`
The default grant is every read plus the safe writes; an unknown scope is refused	`theDefaultGrantIsEveryReadAndTheSafeWritesAndAnUnknownScopeIsRefused`
A write happens only when its confirmation is spent once, by the same connection	`aWriteHappensOnlyWhenItsConfirmationIsSpentOnceByTheSameConnection`
An expired confirmation is refused; the bus still applies the business policy	`anExpiredConfirmationIsRefusedAndTheBusStillAppliesTheBusinessPolicy`
Writes have their own hourly limit	`writesHaveTheirOwnHourlyLimit`
An administrator turns writes off and personal fields on, with policy alone	`anAdministratorTurnsWritesOffAndPersonalFieldsOnWithPolicyAlone`
Blocking a client, ending a person's connections	three tests in the same class
Rate limits per credential and per client	`McpRateLimitIntegrationTest`, `RateWindowTest`
Fixed read boundary; scopes	`McpReadPolicyTest`, `McpScopesTest`
The MCP server's tools, outputs, writes, credentials, HTTP transport	`mcp/tests/server.test.ts`, `writes.test.ts`, `summary.test.ts`, `credentials.test.ts`, `http.test.ts`
The admin screens	`e2e/mcp-admin.spec.ts`, `e2e/mcp-oauth.spec.ts`, `e2e-dispatcher/assistant.spec.ts`, `e2e-store/assistant-access.spec.ts`

11. Frontend tests

Node tests (83 files, 407 tests), grouped:

Group	Examples	Why
Structure	`boundaries.test.ts`	a role never imports another role; shared never imports a consumer
Wording	`wording.test.ts`, `wording-format.test.ts`, `wording-issues.test.ts`, `role-strings.test.ts`, `driver-strings.test.ts`, `loader-i18n.test.ts`, `store-i18n.test.ts`, `thread-strings.test.ts`	one vocabulary, 24-hour depot time, every string in Sinhala and Tamil; `wording-baseline.json` may only go down
Offline	the queue, lanes, retry, retention, kept reads, sync review	the driver's data
Service worker and PWA	`sw-*.test.ts`, `pwa-install.test.ts`, `app-manifest.test.ts`, `push-keys.test.ts`, `assets-size.test.ts`	installability, the offline shell, image budgets
Role logic	`dispatcher-*`, `driver-*`, `loader-*`, `store-*`, `decisions.test.ts`, `inquiry.test.ts`	what each screen computes from the data
Media and speech	`photo-encoder.test.ts`, `taken-photos.test.ts`, `speech.test.ts`, `playlist.test.ts`, `messaging-waveform.test.ts`	photos, voice playback
AI on screen	`dispatcher-explain.test.ts`, `reply-ranking.test.ts`, `reply-templates.test.ts`	the AI-worded text and the ranked replies

Browser suites (80 specs, 249 tests). Each role has its own Playwright config and a mocked API:

- `npx playwright test -c playwright.dispatcher.config.ts`, and the same for driver, loader and store;
- `npx playwright test` for the shell.

They test whole flows on screen:

- the dispatcher plans, decides, publishes and watches Live;
- the driver works offline, records a stop and hands over with the PIN;
- the loader signs in with a PIN, loads and releases;
- the store orders, receives and disputes.

They also test every device size (`devices.spec.ts`), languages (`language.spec.ts`), notifications, messages and voice. In CI the dispatcher suite is split in two shards, with one retry; a failure keeps its report for 7 days.

---

12. How frontend and backend are tested together

The two halves are tested **separately, against a shared contract**:

```
backend                                  frontend
-------                                  --------
contract/ records (views, commands,  <=>  src/shared/domain/*.ts  (hand-kept mirrors)
events) + problem+json
     |                                         |
integration tests prove the backend      browser tests run against mocks.ts,
answers in that shape, with the           written in that same shape
real database                                  |
     |                                   Node tests check the logic on it
ProblemContractTest                      friendly-error.test.ts reads problem+json
CommandPathIntegrationTest                the command envelope in shared/api
```

How the contract is held:

- `contract/` **mirrors.** Every backend contract (view, command payload, event) has a TypeScript mirror in `frontend/src/shared/domain/`, one file per module. A contract change is a change on both sides, in the same pull request (AGENTS.md).
- **The command envelope** is mirrored exactly. `Command.java` says so: "Mirrors `frontend/src/shared/api/commands.ts` exactly, because a device that queued this while offline will replay it verbatim."
- **The error body** (`problem+json` with `violations`) is fixed by `ProblemContractTest`, and `friendly-error.test.ts` reads it.
- **Live checks.** `e2e-loader/live.spec.ts` runs against a real backend when `LOADER_LIVE_PASSWORD` is set. The demo walk is done by hand against the real stack.

The weakness: nothing **automatically** compares the TypeScript mirrors with the Java records. If someone renames a field on one side, the type-checker and the mocked browser tests stay green, and only a live walk finds it. Section 15 has the fix.

---

13. Test doubles: fakes, stubs and recorders

A test double stands in for something real. Waypoint uses doubles only at the edges of the process; the database is always real in integration tests.

Double	Kind	Stands in for	Where
`FakeS3`	fake (works in memory)	the S3 object store	`support/FakeS3.java`
`StubWarehouse`	stub (canned answers)	the external warehouse API	`warehouse/application/StubWarehouse.java`
`RecordingEventPublisher`	spy (records calls)	the outbox, in unit tests	`platform/messaging/RecordingEventPublisher.java` (it has its own test)
A model-serving stub	stub	`ml-server`	in `IntelligenceIntegrationTest`
A fixed `Clock`	fake	real time	every domain test, `TestDates`
`mocks.ts` per role	fake API	the whole backend	browser suites
`TestDatabase`	real, isolated	the production database	`support/TestDatabase.java`: `TEST_DATABASE_URL`, else a container, else skip with the reason

Why not mock the database. The things most likely to break are grants, RLS, isolation and SQL. A mock would agree with whatever the code expects and prove nothing.

14. Redundant, slow or misplaced tests

Every test costs CI minutes and maintenance. These deserve a look.

Test or group	Observation	Suggestion
`planning/bench/OrToolsSpikeTest`, `PeakDayExperimentsTest`, `PriorityReplayTest`	research and benchmarks (engine comparisons, policy experiments, replaying 80 days); they run in the default suite with no tag	tag them `@Tag("bench")` and exclude them from `mvn verify`; run them on demand or weekly
`execution/LiveReadLoadTest`	a load test (1,280 trips) in the default suite	tag it as load; it belongs with the scheduled load tests in EDGE-CASES §12
Overlap between `FoundationIntegrationTest` and `IdentityHardeningIntegrationTest`	both test sign-in, sessions and policies	not truly redundant (the first is the platform promise, the second the hardening), but a failure shows twice; keep both, and name the overlap in their headers
`CommandBusTest` and `CommandPathIntegrationTest`	the same promises at two levels	intended: the unit test proves the logic, the integration test the wiring. Keep
`devices.spec.ts` in driver, loader and store	look alike	different screens per role; not redundant
The planning test that is flaky under load	known from local runs	find its timing assumption; a flaky test teaches people to ignore red
A few domain tests also checked by integration tests	normal in a pyramid	keep, as the unit test is the fast, exhaustive one

No test is clearly useless. The real cost is the benchmark and load tests running on every change.

15. Gaps

#	Gap	Risk	Fix
1	**CI is not running** (GitHub billing)	every guarantee above depends on someone running the suites by hand	restore billing; until then run `mvn verify`, `npm run verify` and the five browser suites before each push
2	**No automatic check that the frontend mirrors match the backend contract**	a renamed field passes every test and breaks in production	generate the TypeScript types from the Java records, or a test that compares JSON samples from integration tests with the mirrors
3	**No automated end-to-end test of the whole day against a real backend**	wiring between real frontend and real backend is checked only by hand	one Playwright suite against `docker compose up` with the seeded demo day: order, plan, publish, load, deliver, receive
4	**23 edge cases have no automated test:** - database failover (PLT-05) and read-replica lag (PLT-06); - catalogue sync down (CAT-01, a chaos drill); - stock changed between check and dispatch (STK-12); - policy authoring with a past date (POL-05); - POL-06 and POL-07 (follow-ups); - the offline ingest load test (EXE-04); - browser storage eviction (EXE-33, manual); - and others	those cases are not proven handled	write the integration ones; schedule the chaos drills and load tests (EDGE-CASES §12)
5	**23 register rows name only a kind** ("Integration", "Unit", "Migration test")	the register cannot be checked against the suite	name the test. Several are tested but unnamed: SEC-08 by `ProblemContractTest`, SEC-14, 18 and 19 by `WarehouseIntegrationTest`, SEC-06 by `IdentityHardeningIntegrationTest`, PLT-01 by the parallel tests, POL-09 by `PolicyCacheTest`
6	**The register says the offline queue retries with jittered backoff (EXE-04), but `backoffMs()` is never called**	a test passes on a function nobody uses	test the queue's actual retry path, then wire it (roadmap F-01)
7	**No property-based tests** (EDGE-CASES §12 plans them for allocation invariants)	generated demand could find a plan that breaks a constraint	add jqwik: for random orders and fleets, every published plan passes `PlanVerification`
8	**No coverage measure** (no JaCoCo)	untested code is invisible	add JaCoCo to `mvn verify` as a report, not a gate
9	**No mutation testing**	tests that run code but assert too little	run PIT on `identity/domain/policy` and `ordering/domain` once, to check the assertions are sharp
10	**Security tests stop at the application**	no scan for known vulnerable libraries, no dynamic scan	add dependency scanning (OWASP dependency-check or GitHub Dependabot) and an occasional OWASP ZAP run on preview
11	**AI: the Jev agreement gate and the TF-IDF baseline are not built**, and there is no labelled evaluation set	the shadow data cannot be judged	build the 400-item labelled set and its evaluation script (roadmap AI-J1)
12	**AI: masking does not cover personal names**, and no test claims it does	a name could reach a provider	add a test with a name in the text, and the dictionary that masks it
13	**Voice recording is tested only with Chromium's fake microphone**	a real phone may behave differently	test on a real Android and iPhone before the pilot
14	**Benchmarks and load tests run on every change** (section 14)	slow suite, CI minutes	tag and exclude
15	**No backup restore test**	durability is unproven	a timed restore drill (roadmap H-05)

16. How to run them

```
# Backend: everything (integration tests need TEST_DATABASE_URL or Docker)
cd backend
JAVA_HOME=/Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/Home \
TEST_DATABASE_URL=postgresql://waypoint:ci-only@localhost:55432/waypoint_test mvn verify

mvn test -Dtest=ModuleBoundaryTest                                    # one class
mvn test -Dtest='CommandPathIntegrationTest#aCommandAppliesOnceHoweverOftenItIsRetried'

# Frontend
cd frontend
npm test                                                              # Node tests
npm run typecheck && npm run build                                    # before any browser run
npx playwright test                                                   # shell suite
npx playwright test -c playwright.dispatcher.config.ts               # a role suite
npx playwright test -c playwright.store.config.ts -g "receives"      # one test by title
npm run verify                                                        # test, typecheck, build, mvn verify

# Model service, MCP, alerts
cd ml-server && pytest
cd mcp && npm test
promtool test rules observability/prometheus/tests/waypoint.test.yml
```

Read the report for skips. An integration test skipped for want of a database is not a pass. Run the full backend suite in a separate git worktree if an IDE is compiling into `target/` at the same time. Never rebuild the frontend while a browser suite is using the build.

---

17. Writing a good test here

1. **Name it as the requirement:** `aStaleVersionIsAConflictAndARetriedCommandIsAnsweredOnce`.
2. **Pick the lowest level that can prove it.** A rule is a unit test; wiring, scope and versions are an integration test; a screen or offline behaviour is a browser test.
3. **For a command, write three cases:** success, one denied scope (`403` plus an audit row), and one stale version (`409`).
4. **Use the injected clock**, never `Instant.now()`. Seed any randomness.
5. **Make your own data:** accounts with a random suffix, a fresh date. Never depend on another test's rows.
6. **Assert the outcome a person sees,** and the database state, not the calls made.
7. **Add the edge-case row and its test together,** and name the test in the register.
8. **For anything external, fake the port,** never the database.
9. **For a flaky test, fix it or delete it.** Never retry it into green.